

OOP Unit 1: Concepts in Python

Task 1: Basic Class Hierarchy (Inheritance)

To avoid rewriting code, I used inheritance. I set up a general Vehicle base class and then extended it into a specific car class, using `super()` to properly initialize the shared traits.

```
class Vehicle:
    def __init__(self, brand, fuel_type):
        self.brand = brand
        self.fuel_type = fuel_type

class Car(Vehicle):
    def __init__(self, brand, fuel_type, num_doors):
        # I am calling the parent constructor to handle brand and fuel_type
        super().__init__(brand, fuel_type)
        self.num_doors = num_doors

# Testing my hierarchy
my_car = Car("Tesla", "Electric", 4)
print(f"My car is a {my_car.brand} ({my_car.fuel_type}) with {my_car.num_doors} doors.")
```

Task 2: Polymorphism with Methods

Here, I wanted different shapes to calculate their areas using their own formulas, while sharing the same interface. I enforced this by using an abstract base class.

```
import math
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * (self.radius ** 2)

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

# Testing my shapes polymorphically
shapes = [Circle(5), Rectangle(4, 6)]
for shape in shapes:
    print(f"Calculated Area: {shape.area():.2f}")
```

Task 3: Encapsulation with Access Control

To protect sensitive data from accidental corruption, I hid the account balance using a double underscore (`__balance`) and provided controlled public methods to interact with it safely.

```
class BankAccount:
    def __init__(self, initial_balance=0.0):
        self.__balance = initial_balance # Private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(self.get_balance())

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient funds or invalid amount!")

    def get_balance(self):
        return f"Current Balance: ${self.__balance:.2f}"

# Managing my account
account = BankAccount(100)
account.deposit(50)
account.withdraw(30)
print(account.get_balance())
```

Task 4: Abstraction with Base Class

I used abstraction to hide the internal implementation details of animal sounds. By creating a template, I forced each specific animal subclass to define its own voice.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

class Dog(Animal):
    def make_sound(self):
        return "Woof!"

class Cat(Animal):
    def make_sound(self):
        return "Meow!"

# Listening to my animals
zoo = [Dog(), Cat()]
for animal in zoo:
    print(animal.make_sound())
```

Task 5: Constructor and Destructor

I wanted to manage the complete lifecycle of an object—from its birth to its removal from memory. I used `__init__` to build it and `__del__` to clean it up.

```
Python
class Person:
    def __init__(self, name):
        self.name = name
        print(f"Hello, I am {self.name}.")

    def __del__(self):
        print(f"Goodbye, {self.name}!")

# Creating and explicitly destroying my object
someone = Person("Alice")
del someone
```

Reflections and Lessons Learnt

Working through these exercises deepened my appreciation for structuring software cleanly. I realized that encapsulation isn't just about hiding data; it's about building a secure perimeter around an object's state to prevent messy bugs. Mastering polymorphism and abstraction showed me how to write highly extensible code—adding new shapes or animals now requires zero changes to the underlying logic. Ultimately, these OOP principles have transitioned my mindset from simply writing functional scripts to architecting resilient, reusable, and highly maintainable software systems.

References

- Lutz, M., 2013. *Learning Python*. 5th ed. Sebastopol: O'Reilly Media.
- Matthes, E., 2023. *Python Crash Course: A Hands-On, Project-Based Introduction to Programming*. 3rd ed. San Francisco: No Starch Press.
- Phillips, D., 2018. *Python 3 Object-Oriented Programming*. 3rd ed. Birmingham: Packt Publishing.
- Ramalho, L., 2022. *Fluent Python: Clear, Concise, and Effective Programming*. 2nd ed. Sebastopol: O'Reilly Media.
- Van Rossum, G. and Drake, F.L., 2009. *The Python Language Reference Manual*. Bristol: Network Theory Ltd.